

Lecture 1

Intro to Intelligent Systems

Intelligent Systems
Vrije Universiteit

1

Hello and welcome to the course Intelligent Systems!

Overview

- 1 Welcome
- 2 Course Introduction
- 3 Discrete Mathematics
 - 3.1 Sets
 - 3.2 Relations
 - 3.3 Graphs
 - 3.4 Trees

Part 1: Welcome

Intelligent Systems
Vrije Universiteit

Who Am I?

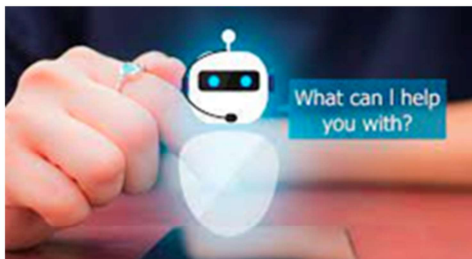
Stefan Schlobach (he/him)

- Studied CS at MPI Saarbrücken and Paris
- PhD in AI from King's College London
- Associate Professor at VU
- Heading the Knowledge in Artificial Intelligence Group
- Knowledge Representation, Semantic Web, Knowledge Graphs, Formal Systems

Part 2: Course Introduction

Intelligent Systems
Vrije Universiteit

Science Fiction?



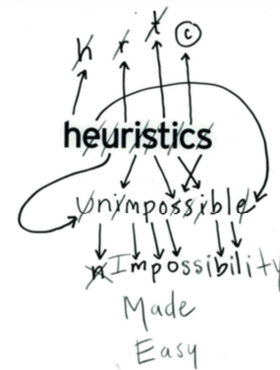
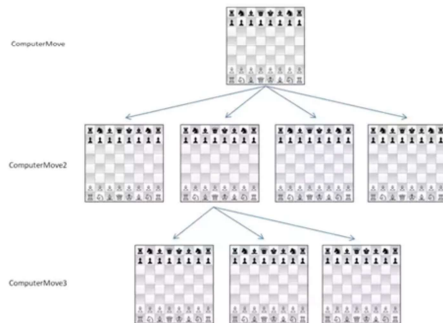
While General AI, i.e. tools that can do any kind of Intelligent behaviour across domains, are still to be considered “science fiction”, a significant part of specific AI functionality is already in existence. Alpha Go has recently shown that computers can play even the most complex strategy games (such as GO) better than humans, the self-driving car shows that AI systems can operate in uncertain environments and adapt to complex sensoric information. Siri is one of the interfaces of i-phones, which allows users to ask open questions in Natural Language. The IBM system Watson, finally, has shown to be able to answer complex open domain questions.

So no, AI and Intelligent Systems is no science fiction.

Part 2.1: Core Topics

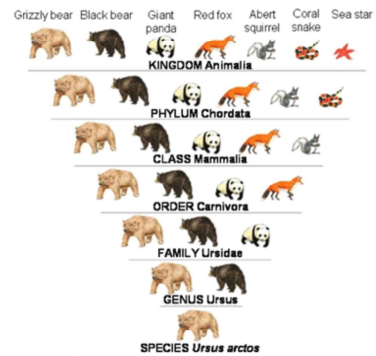
Intelligent Systems
Vrije Universiteit

Problem Solving



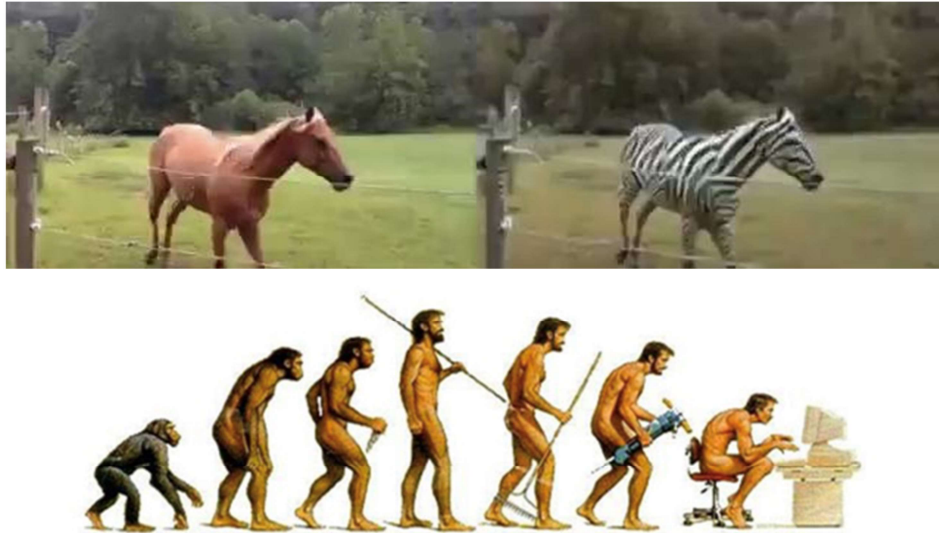
In the first two weeks of this course we focus on problem solving, or the process of finding the best solution to a problem. One example of this is **choosing the best move** in a game of chess out of all possible moves, even looking ahead more than one move to predict the outcome. **Heuristics** are one method of problem solving used by intelligent agents which we will cover next week.

Knowledge & Reasoning



We will look into knowledge and reasoning, or how we can represent information in a structured way. Hierarchies of information are crucial for the functioning of artificial intelligence.

Adaptivity & Learning



10

In the last weeks of the course we focus on adaptivity and learning, in lectures 1-3 on machine learning. Learning requires the combining of structured information gained from experience with problem solving methods.

In the video above machine learning is used to make a horse look like a zebra in motion. This is done by learning what is called latent features of being a zebra (namely having stripes) and propagate this on the video of a horse. You can train Neural Networks to be able to do this. Simpler applications of machine learning are the classification of emails as spam or regular mail. We will be looking into how to build a simple classifier such as this towards the end of the course, and we will look into how to learn strategies to play a card game.

But there are other ways of adaptation as well. The VU also has a research group for evolutionary methods for adapting a machine to perform better.

Part 2.2: Schnapsen as a Learning Tool

Intelligent Systems
Vrije Universiteit

11

This course is unique in that we use the card game Schnapsen as a theme through which to learn simple AI techniques. In this course we will build an AI that can play Schnapsen.

Schnapsen

Popular [card game](#) in Austria, southern Germany, and Hungary.

- Similar to 66
- Etymology: Schnappen not Schnaps

Trick-based card game, whoever wins starts the next trick.

[Goal](#): collect 66 points.

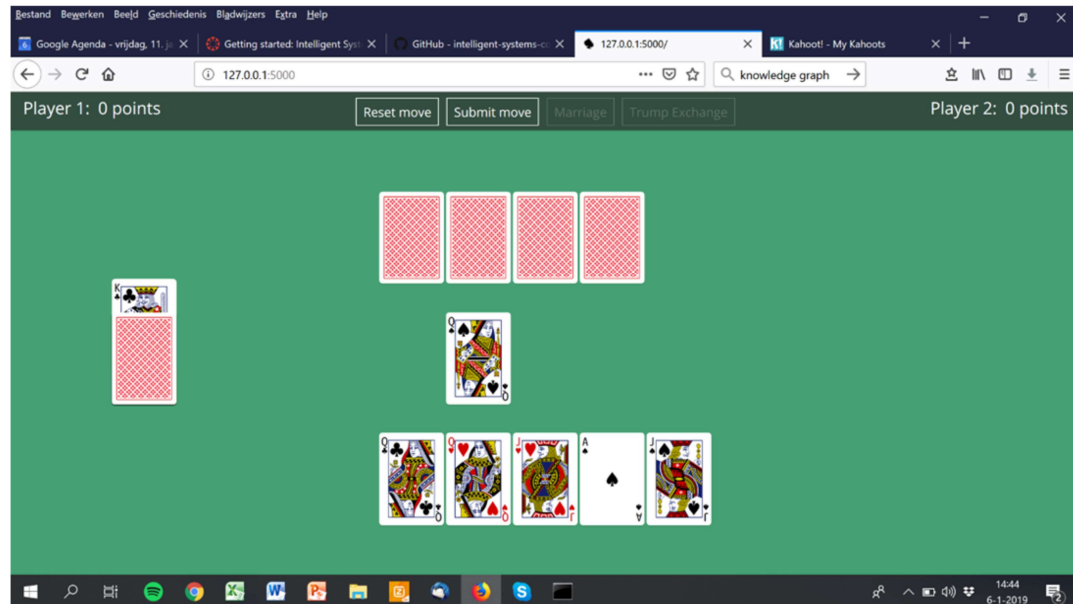
- Tallon or stock

Card	Points
Ace	11
Ten	10
King	4
Queen	3
Jack	2

Try it yourself: [Schnapsen platform on Canvas](#)

There are only twenty cards we use in the game of Schnapsen. Each player has five cards in their hand, and there is a stack of ten cards in the center. In the first phase of the game, which lasts while the deck (talon) in the middle is not empty, the player wins the trick if they play a higher card. In the second phase of the game, the players are required to follow suit of the previous card when they play a card.

Schnapsen



13

This is a Schnapsen platform built by a former student which you can download in order to practice the rules of the game. At the end of the course you will try to program a player which can beat this platform, and Project Intelligent Systems is about evaluating the impact of

Schnapsen Variants

Simpler versions exist, like Schnaptience and mini Schnapps



SINGLE PLAYER VARIANT

Simpler versions can be easier to computationally understand and we will start to program a Schnapsen-playing agent using one of these simpler versions.

Properties of Schnapsen

- Adversarial agent setting
The opponent interferes with your objective.
- Imperfect information game
Much uncertainty, e.g. what cards does the opponent have?
- 2 game phases
- Explicit knowledge available

What Google did for Go, we will do for Schnapsen.

15

Games are an **adversarial** setting, meaning I have an adversary which has a different goal than I do. That means the other agent's actions interfere with your ability to reach your goal. It is an imperfect information game because we don't know the order of the cards in the deck in the first phase of the game.

We will focus first on the theory and some programming basics, and do some research and development in January.

Part 2.3: How to Pass the Course

Intelligent Systems
Vrije Universiteit

Course Overview

A generic intelligent systems framework...

Efficient, simple, correct, well- understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Maths: discrete math (logic, sets, graphs, trees), statistics (probability), linear algebra, etc.

17

We will focus on the three core topics, problem solving , knowledge and reasoning , and learning and adaptivity. At the end of each lecture I will update you on our progress in the IS framework.

Today we will start by looking at some simple mathematical concepts of sets.

How to Pass

1. Check Canvas regularly
2. Participate in work groups on-campus and complete the exercises
 - a. 6 attendances = bonus
3. Attend lectures
4. Complete the 11 **weekly assignments** (30% of total)
 - a. Register for programming support as needed in the Google sheet
5. Pass the digital **midterm and final exam** (70% of total)
 - a. Theory from lectures
 - b. Application questions from exercises in work group

Today: Some mathematical foundations: Basic Discrete Mathematics

18

All of the information on the Canvas home page is the most accurate information; you can find this information there as well.

What is the difference between the work groups and the practicals? Work groups are meant for practicing the theoretical material and exercises that are posted weekly with your TA; this is the material that is crucial for the exams but does not need to be submitted. The practicals are meant for getting help with the weekly programming assignments, which you must submit for a grade, from a TA.

Part 3: Discrete Mathematics

Intelligent Systems
Vrije Universiteit

19

Today we will cover some set theory so that you can understand the different symbols and terms we'll start to use. A widely agreed on vocabulary is crucial for formalising concepts for computers.

3. Discrete Mathematics

3.1 Sets

3.2 Relations

3.3 Graphs

3.4 Trees

Expectations for Today

There is not much to understand today.

All we are going to do is **to define** things.

Today's lecture does not contain anything difficult to understand. All we do is to **define** notions, often abbreviations and shorthands.

There is a huge amount of material about this, if you find this slide-deck not helpful, go out and look for basic material on Sets, Relations, Graphs, Trees and other basics of Discrete Mathematics.

Part 3.1: Naive Set Theory

Intelligent Systems
Vrije Universiteit

22

Today we will cover some set theory so that you can understand the different symbols and terms we'll start to use. A widely agreed on vocabulary is crucial for formalising concepts for computers.

Set theory is more complex than one might think, things become tricky once you allow sets to contain sets. Therefore, the mathematics we introduce here are called Naive Set Theory.

Lists and Sets

A **set** is a structure representing an *unordered* collection of zero or more *different* objects.

Set theory deals with operations between, relations among, and statements about sets.

Notation: { }

A **list** is a structure representing an *ordered* collection of zero or more objects which may include *duplicates*.

Notation: []

23

List and set are two ways to describe a group of objects. The key difference is that a **set** { } is an *unordered* collection of *different* objects, while a **list** [] is an *ordered* collection of objects which *may have duplicates*.

Basic Set Notation

For sets we use variables, *e.g.* S , T , U , etc.

We denote a set S in curly brackets with all variables inside: $\{S, T, U\}$

Set builder notation: for any proposition $P(x)$ (statement), $\{x/P(x)\}$ is the set of all x such that $P(x)$.

$$\begin{aligned} \text{e.g. } \{x \mid x \text{ is an integer where } x > 0 \text{ and } x < 5\} \\ = \{1, 2, 3, 4\} \end{aligned}$$

24

We denote objects in a set with variables inside of curly brackets. We can define sets with propositions, which are statements that are true about the set, denoted with $P(x)$.

For example, let's say $P(x)$ = all integers between 0 and 5. If we want to define this set, we can note

$$\{x \mid x \text{ is an integer where } x > 0 \text{ and } x < 5\}$$

We can read this statement as "x such that x is an integer where x is greater than 0 and less than 5".

Basic Properties of Sets

Elements in a set are...

1. Unordered

$$\{a,b,c\} = \{b,c,a\} = \{c,b,a\}$$

= etc.

1. Distinct (unique)

$$\{a,b,c\} = \{a,a,b,b,c,a,b,a\}$$

Two sets are **equal** if and only if (*iff*) they contain exactly the same elements.

It does not matter how the set is defined, *e.g.*

$$\{1,2,3,4\} =$$

$$\{x \mid x \text{ is integer between } 0 \text{ and } 5\} =$$

$$\{x \mid x \text{ is pos. Integer whose square is between } 0 \text{ and } 25\}$$

Since objects in sets are unordered and distinct, two sets are equal if and only if (*iff*) they contain exactly the same elements. This is not the case for lists.

Infinite Sets

Sets may be never ending ([infinite](#)):

$$\mathbf{N} = \{0, 1, 2, \dots\}$$

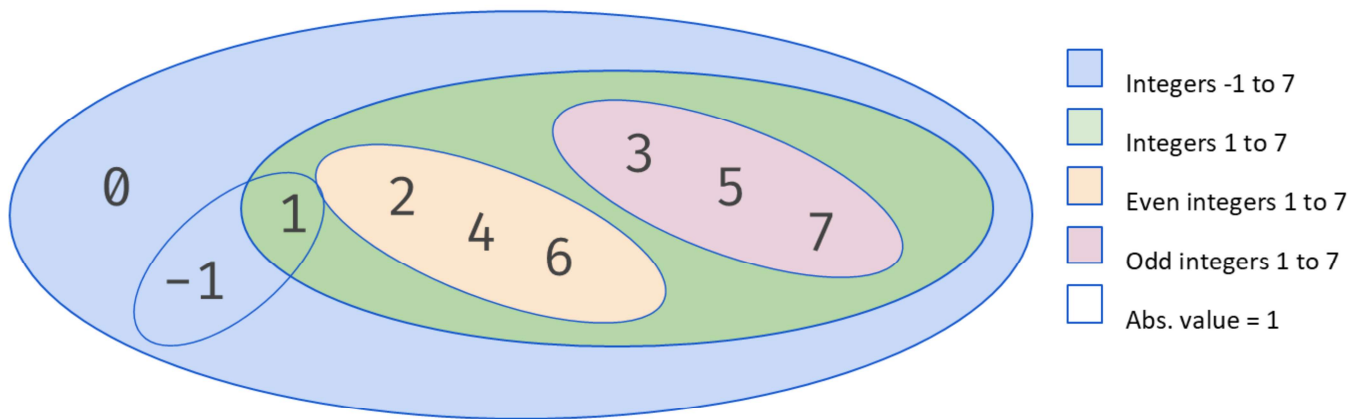
$$\mathbf{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$$

$$\{\mathbf{R} \mid \mathbf{R} \text{ has less than 4 decimal places}\} = \{1.0, 1.0001, 1.001, \dots\}$$

Sets do not have to be finite. For example, the set of all natural numbers, all integers, or all real numbers are all infinite.

Venn Diagrams

We can model sets using Venn Diagrams



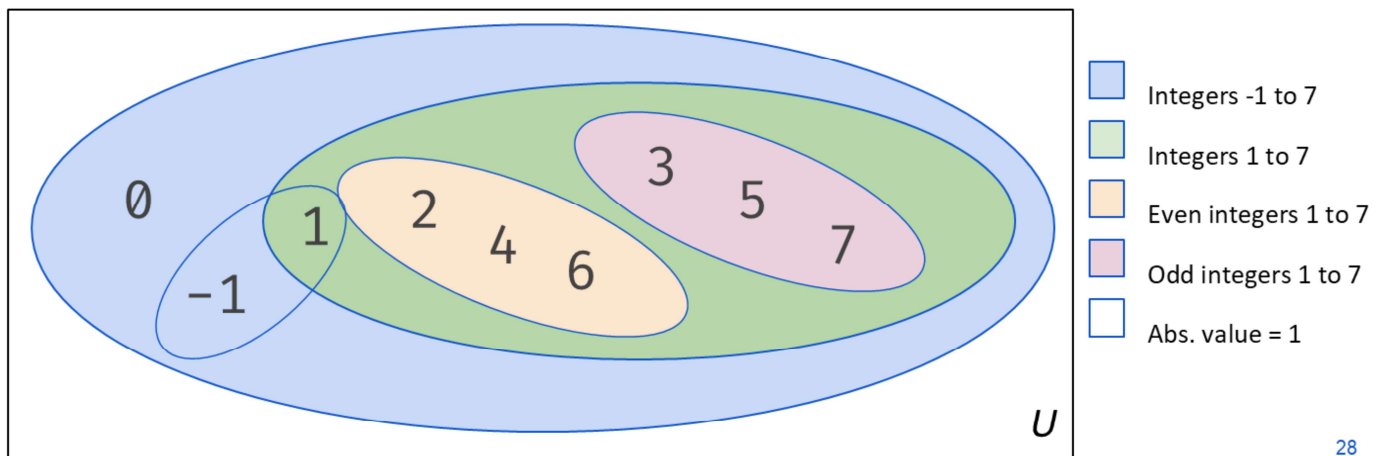
27

Venn diagrams are useful tools for visualising sets and their members.

The blue circle represents all integers -1 to 7. Within that set we also have integers 1 to 7 and the set of numbers whose absolute value is 1. We can further specify sets within those that become increasingly specific.

The Universe

The **universe** (of discourse) U is the set of all things.



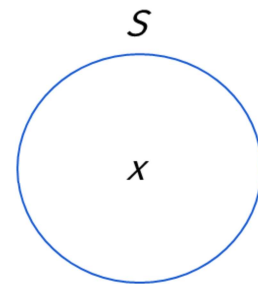
28

The concept of a **universe** is the set of all things. In Venn diagrams of sets we usually denote the universe with a square. This is useful because it allows us to make statements like, "all the objects in the universe outside of the blue circle," which in this case is none.

Set Membership

$x \in S$ means “ x is an **element** or **member** of set S ”.

- “ x is in S ”.



We can define set equality in terms of the $\in$ relation:

Set A is equal to set B *iff* they have the same exact members.

$x \notin S$ *iff* x is not in S .

Examples stefan $\in$ Teachers
 boeing 747 $\in$ Airplanes

The Empty Set

$\emptyset$, the null or **empty set**, is the unique set that contains no elements.

$$\emptyset = \{\} = \{x \mid \text{False}\}$$

No matter the domain of discourse, we have the axiom

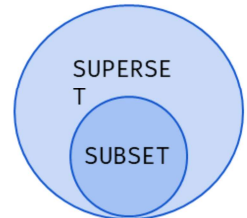
$$\neg \exists x : x \in \emptyset$$

Example set of all people who are 5 meters tall = $\emptyset$

Subset and Superset

$S \subseteq T$ means “ S is a **subset** of T ”, thus every element of S is also an element of T .

- $S \subseteq T \Leftrightarrow \forall x (x \in S \rightarrow x \in T)$
- $\emptyset \subseteq S, S \subseteq S$

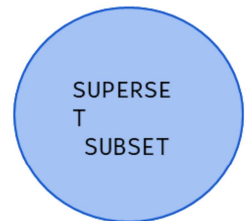


$S \supseteq T$ means “ S is a **superset** of T ”, thus $T \subseteq S$

- Every element in T is in S , but not necessarily vice versa

OR THEY MAY BE EQUAL

Note: $S = T \Leftrightarrow S \subseteq T$ and $S \supseteq T$



31

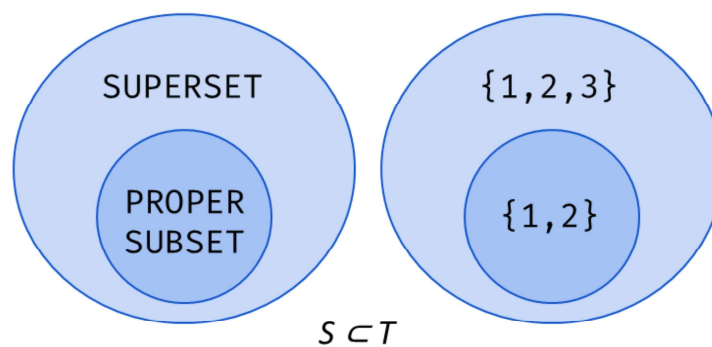
Examples mammals $\subseteq$ animals
 animals $\supseteq$ mammals

But also...
 mammals $\subseteq$ mammals

Proper (Strict) Set

$S \subset T$ means “ S is a **proper subset** of T ”, thus $S \subseteq T$ but $T \not\subseteq S$.

In other words, every element of S is in T , but not every element of T is in S .



32

Example $\{1, 2\} \subset \{1, 2, 3\}$

However

$$\{1, 2\} \not\subset \{1, 2\}$$

Sets as Objects in Sets

The elements in a set may be sets themselves.

e.g. $\{\{1,2\},\{3,4\}\}$

e.g. $S = \{x \mid x \subseteq \{1,2,3\}\}$

$S = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$

Important note: $1 \neq \{1\} \neq \{\{1\}\}$

A set can also be an object in itself. So T is an object but $\{1,2\}$ may also be an object, meaning we can place it inside another set.

Example we have a set called S where $S = \{\{1,2\},\{3,4\}\}$. How many objects are in S ? 2.

Cardinality

$|S|$ means “the **cardinality** of S , or the measure of how many elements S has.

$$\text{e.g. } |\{1,2,3\}| = 3$$

$$|\emptyset| = ?$$

$$|\{5,7,9\}| = ?$$

$$|\{1, \{1,\{2,3\}\}| = ?$$

We say that a set is **finite** if it is not infinite.

34

Cardinality of a set is the number of objects in the set. *Examples*

$$|\{1,2,3\}| = 3$$

$$|\emptyset| = 0$$

$$|\{\{5,7,9\}\}| = 1$$

$$|\{1, \{1,\{2,3\}\}| = 2$$

Power Set

The **power set** $P(S)$ of a set S is the set of all subsets of S .

$$P(S) = \{x \mid x \subseteq S\}$$

$$\text{e.g. } P(\{a,b\}) = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}$$

Sometimes $P(S)$ is written as 2^S . Note that for finite S , $|P(S)| = 2^{|S|}$.

Weird observation: there are different sizes of infinite sets!

$$|P(\mathbf{N})| > |\mathbf{N}|$$

35

The **power set** $P(S)$ of a set S is the set of all subsets of S . $P(S) = \{x \mid x \subseteq S\}$. *Example*

$$P(\{a,b\}) = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}.$$

The power set $P(S)$ always contains the empty set $\emptyset$

Sometimes $P(S)$ is written 2^S .

Note that for finite S , $|P(S)| = 2^{|S|}$.

It turns out that $|P(\mathbf{N})| > |\mathbf{N}|$. *There are different sizes of infinite sets!*

Union Operator

$A \cup B$ means “A **union** B”, or the set containing all elements that are either in A or (“V”) in B. Formally, $A \cup B = \{x \mid x \in A \vee x \in B\}$.

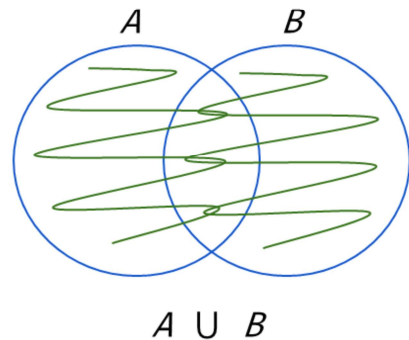
e.g. $A = \{1, 2, 3\}$

$B = \{4, 5, 6\}$

$A \cup B = \{1, 2, 3, 4, 5, 6\}$

$A \cup B$ is all elements in A and all elements in B:

$$\forall A, B: (A \cup B \supseteq A) \wedge (A \cup B \supseteq B)$$



For sets A, B , their **union** $A \cup B$ is the set containing all elements that are either in A, **or** (“V”) in B, or in both. Formally, we can say that the union of A and B is a such that a is in A or a is in B.

Example $A = \{\text{birds}\}$ and $B = \{\text{reptiles}\}$, then A union B is All birds plus All reptiles.

Intersection Operator

$A \cap B$ means “A **intersection** B”, or the set containing all elements that are both in A and (“ $\wedge$ ”) in B. Formally, $A \cap B = \{x \mid x \in A \wedge x \in B\}$

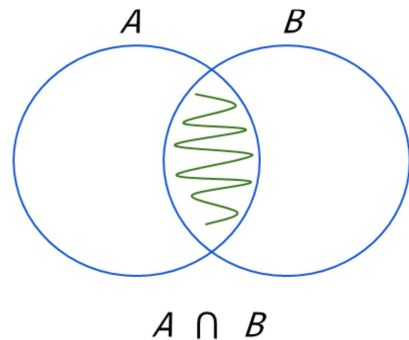
e.g. $A = \{1, 2, 3\}$

$B = \{3, 4, 5\}$

$A \cap B = \{3\}$

$A \cap B$ is a subset of A and a subset of B:

$$\forall A, B: (A \cap B \subseteq A) \wedge (A \cap B \subseteq B)$$



37

For sets A, B , their **intersection** $A \cap B$ is the set containing all elements that are either in A **and** (“ $\wedge$ ”) in B. Formally, we can say that the intersection of A and B is a such that a is in A and a is in B.

Example $A = \{\text{birds}\}$ and $B = \{\text{reptiles}\}$, then $A \text{ intersection } B = \{\emptyset\}$.

Disjointness

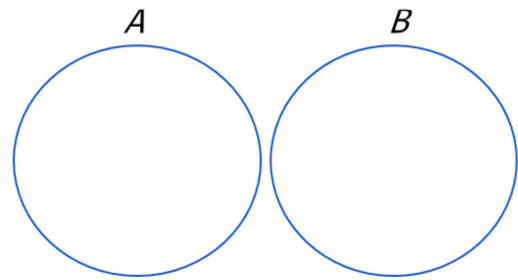
$A \cap B = \emptyset$ means “A and B are **disjoint**”, or their intersection is empty.

e.g. $A = \text{“even integers”}$

$B = \text{“odd integers”}$

e.g. $A = \{1, 2, 3\}$

$B = \{4, 5, 6\}$



$$A \cap B = \emptyset$$

Disjoint sets are two sets that have *no objects in common*.

Example birds and reptiles are disjoint sets.

Birds and yellow animals are not disjoint sets (e.g. canary).

Set Difference

$A - B$ means “the **difference** of A and B ”, or the set of all elements in A but not in B .

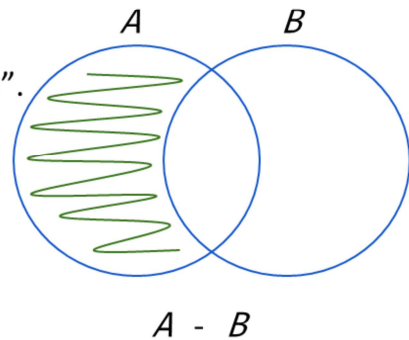
$$A - B = \{x \mid x \in A \text{ and } x \notin B\}$$

Also called “the *complement* of B with respect to A ”.

e.g. $A = \{1, 2, 3\}$

$$B = \{3, 4, 5\}$$

$$A - B = \{1, 2\}$$



Example $A = \{\text{birds}\}$, $B = \{\text{yellow animals}\}$

$$A - B = \{\text{birds that are not yellow}\}$$

The complement of B w.r.t $A = \{\text{birds that are not yellow}\}$

Set Complements

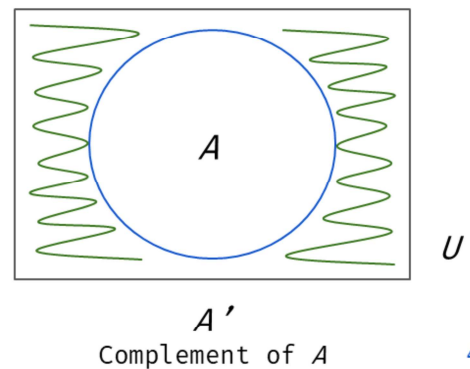
Recall that the universe U is the set of all things.

A' is “the **complement** of A ”, or everything in the universe that is not A . Also written as $U - A$ or A with respect to U .

e.g. $U = \text{“positive integers”}$

$A = \{3, 4, 5\}$

$A' = \{1, 2, 6, 7, \dots\}$



40

When we are talking about the **complement** of a set with respect to the universe, we can simply note this with an apostrophe on the name of the set.

Example The complement of A with respect to $U = A'$

$U = \{\text{animals}\}$

$A = \{\text{birds}\}$

$A' = \{\text{all animals that are not birds}\}$

Counting Elements

How many elements are in $A \cup B$?

Idea: Add the cardinality (no. of elements) in A and B

$$|A \cup B| = |A| + |B|$$

Problem: we're double-counting any elements that in both A and B

Solution: subtract the intersection of A and B

$$|A \cup B| = |A| + |B| - |A \cap B|$$

e.g. how many elements are in $\{1,2,3\} \cup \{3,4,5\} = ?$

Answer: 5 elements

Set Equivalences

Identity	$A \cup \emptyset = A$	$A \cap U = A$
Domination	$A \cup U = U$	$A \cap \emptyset = \emptyset$
Idempotent	$A \cup A = A = A \cap A$	
Double complement	$A'' = A$	
Commutative	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associative	$A \cup (B \cup C) = (A \cup B) \cup C$	
	$A \cap (B \cap C) = (A \cap B) \cap C$	
De Morgan	$(A \cap B)' = A' \cup B'$	$(A \cup B)' = A' \cap B'$

42

This is a useful list of set equivalences to consider. We can use set theory to make calculations. These rules are trivial when you understand the concepts and you shouldn't need to memorise these, rather understand the concepts. Those calculation rules are often called an Algebra.

Instead of a final slide:

- Next lecture tomorrow (come)
- Register with a working group
- Learn Schnapsen

Part 3.2: Relations

Intelligent Systems
Vrije Universiteit

Lists

A **list** is an ordered sequence (an *array*) of elements

Examples

DigitList := $\langle 0, 1, 2, \dots, 9 \rangle$

LetterList := $\langle a, b, c, \dots, z \rangle$

MixedList := $\langle t, h, u, \text{ }, f, e, b, \text{ }, 1, 1 \rangle$

- ▶ elements in a list have **positions** $\langle 1, 2 \rangle \neq \langle 2, 1 \rangle$
- ▶ finite lists have a **length** $\langle 1, 2 \rangle \neq \langle 1, 2, 1 \rangle$

t	h	u		f	e	b		1	1
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
1	2	3	4	5	6	7	8	9	10

Cartesian Product

Cartesian product of two sets

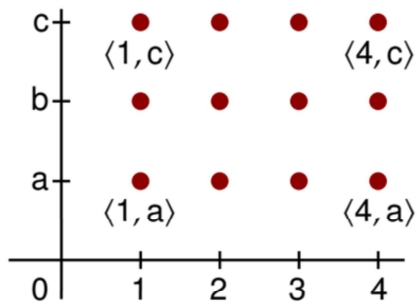
$$A \times B := \{\langle a, b \rangle : a \in A \text{ and } b \in B\}$$

$$A \times A =: A^2$$

set of **pairs**

(lists of length 2)

Example



Cartesian product
 $\{1, 2, 3, 4\} \times \{a, b, c\}$

N-tuples and relations

N-tuples are sequences of lengths n , whose order matters $(a_1, a_2, a_3, \dots, a_n)$

(a) is a **singlet**

(a,b) is a **pair**

(a,b,c) is a **triple**

Relations are **sets of n-tuples**, e.g.

$\text{likes} = \{(\text{john}, \text{mary}), (\text{john}, \text{cars}), \dots, (\text{mary}, \text{apples})\}$

In other words, subsets of the Cartesian product $\{\text{john}, \text{mary}\} \times \{\text{mary}, \text{cars}, \text{apples}\}$

47

What we started out with was this collection of things that we keep in sets. Sets are unordered collections of elements that were unique; now we look at a different type of concept, a tuple where we have a specific order in the sequences of the elements that we want to deal with. Note that an n -tuple with only one element is still surrounded by $()$. A 2-tuple is called a pair, a 3-tuple is called a triple, and further n -values are simply referred to as 4-tuple, 5-tuple, and so on. When we talk about a set we refer to unordered unique objects; tuples invoke an order; relations are collections of tuples that all relate to each other in the same manner.

Relations

A **relation** R of **type** $A_1 \times \dots \times A_n$ is a subset of $A_1 \times \dots \times A_n$

Example

Thu, Feb 11, 2016

Time	Platform	Destination	Train
11:00	4	Nijmegen	Intercity
11:03	1	Uitgeest	Sprinter
11:07	4	Rhenen	Sprinter
11:14	1	Schagen	Intercity

Relation of type $Dates \times Times \times Platforms \times Stations \times TrainTypes$

48

Relations are at the core of current Database systems, which are often also called Relational Databases. Data is stored as a Relation, attributes are types, etc. Based on some logical rules on sets and relations, a so called Relational Algebra is implemented over those Databases, which allows us to write Database queries in a well-understood manner.

Binary relations and infix notation

A **binary relation** is a relation of type $A \times B$ or A^2

A relation **in the set** A is a binary relation of type A^2

Examples

$IsBrotherOf := \{\langle x, y \rangle : x \text{ is brother of } y\} \subseteq People^2$

$FollowsCourse := \{\langle x, y \rangle : x \text{ follows } y\} \subseteq Students \times Courses$

Alternative infix notation

$x R y$ instead of $\langle x, y \rangle \in R$

$x IsBrotherOf y$ instead of $\langle x, y \rangle \in IsBrotherOf$

$x FollowsCourse y$ instead of $\langle x, y \rangle \in FollowsCourse$

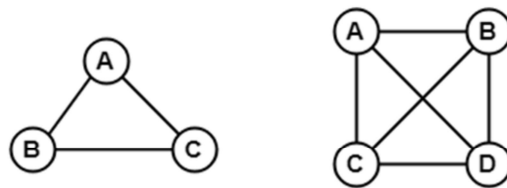
Part 3.3: Graphs

Intelligent Systems
Vrije Universiteit

Graphs

A **graph** G is mathematical generalisation of the simple concept of a set of dots, links, edges, or arcs.

$G = (V, E)$, where V is the set of **vertices/nodes** (singular: vertex), and E is the set of paired vertices or **edges**.

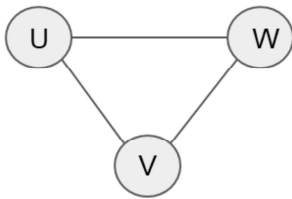


The concept of graphs is omnipresent in computer science. We can use graphs to represent the concepts of dots, links, edges, or arcs. A graph is written in (V, E) notation, where V is the set of vertices and E is the set of paired vertices or edges. Here are two simple examples of how graphs are drawn.

Types of Graphs

undirected (simple)

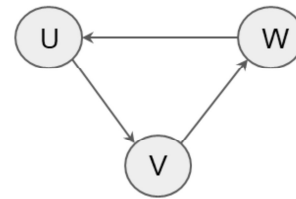
A non-empty set of vertices V , and a set of **unordered**, distinct edges E .



$G(V,E)$ $V = \{u,v,w\}$
 $E =$
 $\{\{u,v\},\{v,w\},\{u,w\}\}$

directed

A non-empty set of vertices V , and a set of directed edges E , consisting of **ordered** pairs.



$G(V,E)$ $V = \{u,v,w\}$
 $E =$
 $\{(u,v),(v,w),(u,w)\}$

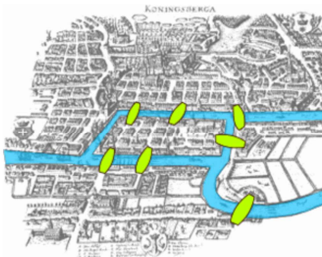
52

There are two important distinctions in types of graphs, namely the undirected (simple) and the directed graph. Undirected graphs include a set of vertices and a set of edges which are unordered and distinct. The set of edges has unordered sets inside of it, written with curly brackets, indicating which two nodes are connected; they can be written in any order within the interior curly brackets $\{\}$. Directed graphs include a similar set of vertices, however its set of edges is ordered and distinct. Within the relation E there exist n -tuples which specify which two nodes are connected but also the direction of their relationship to one another. The n -tuples are distinguished by round brackets $()$.

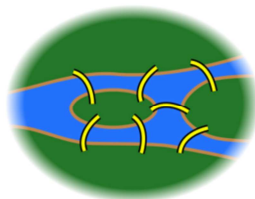
Solving Problems with Graphs

The Seven Bridges of Königsberg (Euler's problem)

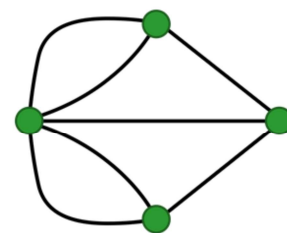
The residents of Königsberg, now Kaliningrad, want to find a walking route which crosses each of the seven bridges over the Presel river exactly once, ending up at the starting point.



7 bridges over Presel



Original Abstraction

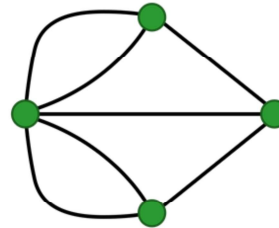
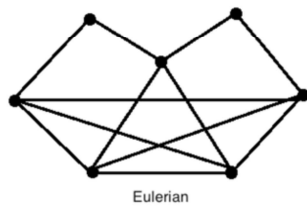


Redrawn Abstraction

Graph theory started with Euler several hundred years ago, who studied a specific task in his hometown, Königsberg (now Kaliningrad). The problem described can be solved using an abstract mathematical representation and general properties to prove or disprove that such a path exists across the seven bridges. When we abstract the environment we have an undirected graph; when we create a walking route that visits each node once we are left with a directed graph that can answer our original query.

Euler (illustrative, not for exam)

Def: A graph is **Eulerian** if it is both connected and has a closed trail (a walk with no repeated edges) containing all edges of the graph.



Theorem: A graph is **Eulerian** if every node in the graph has an even degree.

Applications: computer networks, shortest distance problem, assign channels to TV stations, state-space representations, knowledge graphs

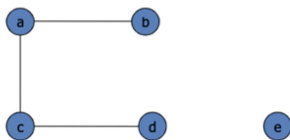
We can use mathematics to solve practical problems in real life. What Euler did at the time was that he defined the notion of traversability a path in a graph that uses each node precisely once. It's defined mathematically as a graph with exactly two vertices of off. The important idea is that applying a mathematical idea, in this case a specific function to an *undirected* graph, has many practical applications.

Alternative representations for graphs

The **adjacency matrix** of a graph G is the matrix A defined as adjacency between nodes u and v ;

- if the set of edges E includes (u,v) , then $A[u,v]$ has a value of 1
- if there is no direct path u to v , then $A[u,v]$ has a value of 0

$$A[u,v] = \begin{cases} 1 & \text{if } uv \in E_G, \\ 0 & \text{otherwise.} \end{cases}$$



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	1	0	0
<i>b</i>	1	0	0	0	0
<i>c</i>	1	0	0	1	0
<i>d</i>	0	0	1	0	0
<i>e</i>	0	0	0	0	0

55

We have established that graphs can be represented as sets; a pair of 2-to-2 sets. They can also be represented by a diagram as shown on the left, but also as a matrix shown on the right. Such matrices are called adjacency matrices, which are written with all the nodes in the graph across the first row and down the first column. The order in which they are written is arbitrary, but the same order must be upheld going left-to-right on the first row as top-to-bottom in the first column. When there is a direct path between two nodes, a 1 is written in the cell, otherwise a 0 is written. Adjacency matrices are very powerful when working with graphs and extracting information because machine learning largely functions on these kinds of structures and linear algebra.

Part 3.4: Trees

Intelligent Systems
Vrije Universiteit

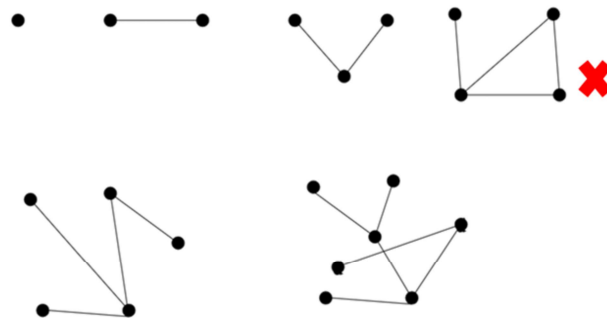
56

The following section concerns trees, a type of graph, and their terminology.

Trees

A **tree** is a connected, undirected graph with no simple circuits.

A **circuit** (cycle) is a path of length ≥ 1 that begins and ends at the same vertex.



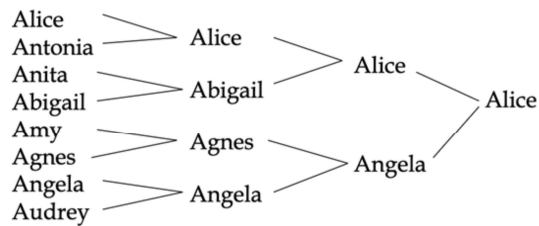
57

A tree is a type of connected, undirected graph which does not contain any simple circuits. A simple circuit is defined as a path in a graph with a length of more than 1 which begins and ends at the same vertex; these are identifiable as loops. All the examples on the slide are trees except for the rightmost graphs. Both of these contain a simple circuit.

Tree examples

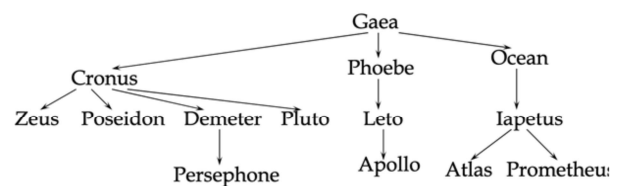
Tournaments

Describes outcome of a series of games.



Family tree

Common terminology is derived from family trees.

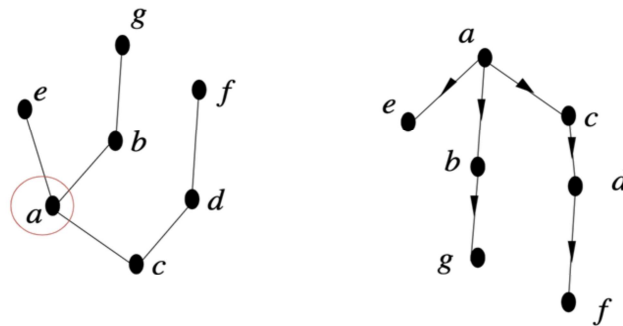


There are many examples of trees that we use in daily life. In tournaments, trees are used to display which two teams play each other in each game, as well as the outcome of that game. In this example we have a tennis team with eight teams, or players, where the winner proceeds.

Family trees are the source of common tree terminology used in graph theory today, because thinking of trees in terms of familial relations gives us intuitive terminology.

Rooted trees

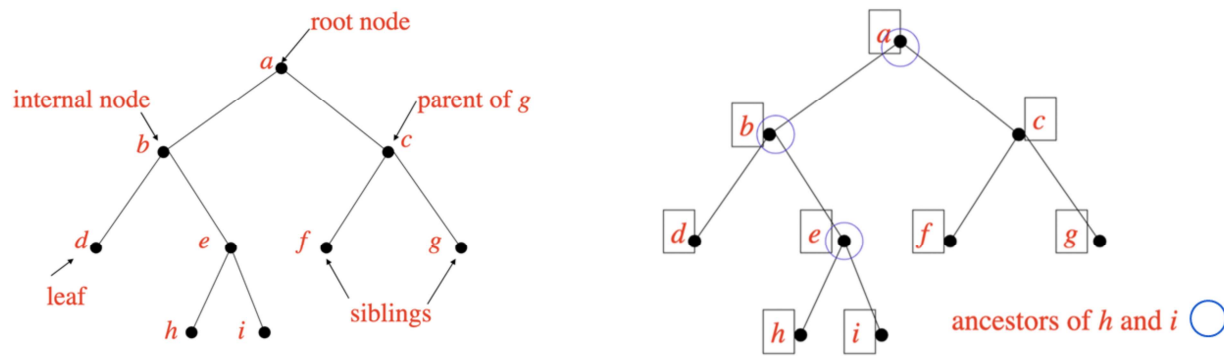
Once a tree has a designated **root** node, it is possible to assign a direction to each edge.



59

In an undirected graph, we can choose to designate any node as the root node and turn the tree such that the root node appears above all other nodes. In this example, *a* is designated as the root node and the tree is redrawn to reflect this, without removing or adding any nodes or edges. Only now can we add directionality to a tree; all nodes can be reached from *a*, but not the other way around.

Tree terminology



60

A *parent node* is any node which has *children*, which are nodes that appear below it in a tree and can be directly expanded towards, or reached, from the parent. This node is only called the *parent* when there is direct link to the child using one edge, not an extended path.

An *internal node* is one which has both a parent and child(ren). In contrast, *leaf nodes* have a parent but no child(ren). Nodes are *siblings* when they share the same parent node.

Ancestors to a node are all nodes which must be passed through when drawing the shortest path to the root. In this example, the path from *h* and *i* to the root node *a* must pass through the nodes *e*, *b*, and *a*.

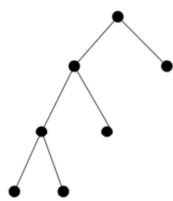
Moving forward, trees will be drawn as they appear above, undirected and without arrows.

N-ary trees

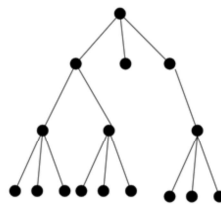
A rooted tree is an **n-ary** tree if every internal vertex has no more than n children.

A tree is **full n-ary** if every internal vertex has exactly n children.

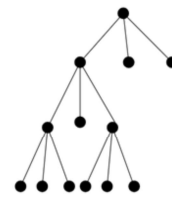
An n -ary tree with $n = 2$ is called a **binary tree**.



full 2-ary (binary)



3-ary (ternary)



full 3-ary (ternary)

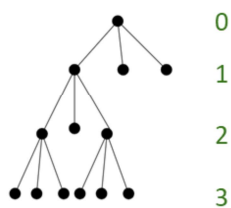
We can further categorise trees by the number of children each *internal* node has.

Recall that an internal node is one which has children. If every internal node of a tree has maximally two children, it is called a binary tree; for maximally three children we say tertiary tree, and so on.

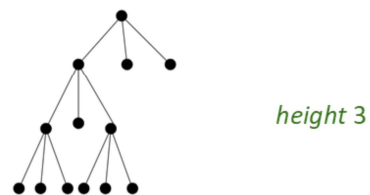
A tree is a full n -ary, where n is the number of children for each internal node, is every internal node has exactly n children.

Properties of trees

The level or **depth** of a vertex v in a rooted tree is the length of the unique path from the root to this vertex.



The **height** (also called depths) of a rooted tree is the maximum of the levels of vertices.



The *depth* of a tree is defined as the length of the unique path from the node to the root. As a result, the root node is at depth 0. The height of the tree is maximum depth; the depth of the node furthest from the root.